Carrera: Ingeniería en Sistemas de Información

Comisión: 3EK02

Asignatura: Tecnologías de Desarrollo de Software IDE

TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión Académica

Grupo N°05	**Fecha: 26/10/2026**
Docentes: • **Ing. Ezequiel Porta** • **Lic. Sebastián Rocco**	
Integrantes: • **Tosar, Salvador — Legajo: 53914 (salvadortosar@gmail.com)** • **Yocco, Simón — Legajo: 55263 (yoccosimon@gmail.com)**	
Repositorio Oficial: https://github.com/Simon-Yocco/Academia.git	

Índice

1. Introducción y Descripción General del Sistema 2
2. Requerimientos del Sistema 2
2.1. Requerimientos Funcionales (RF) 2
2.2. Requerimientos No Funcionales (RNF) 4
3. Arquitectura del Sistema 5
3.1. Detalle de Proyectos de la Solución 5
4. Modelo de Datos 7
4.1. Modelo de Dominio (Diagrama de Clases UML) 7
4.2. Normalización y Relación Persona - Usuario 8
4.3. Diccionario de Datos 8
5. Patrones de Diseño y Decisión Técnica de Persistencia 9
5.1. Implementación Obligatoria de ADO.NET Clásico 9
5.2. Persistencia Principal con Entity Framework Core 8 9
5.3. Patrón Maestro/Detalle (Planes de Estudio y Materias) 9
6. Tecnologías y Herramientas Utilizadas 9
7. Datos de Prueba y Manual de Credenciales 11
7.1. Administrador 11
7.2. Profesores (Docentes) 11
7.3. Alumnos 11
8. Evidencias de Ejecución 12
8.1. Aplicación de Escritorio (Windows Forms) 12
8.2. Aplicación Web (Blazor WebAssembly) 14
9. Conclusiones y Enlaces del Repositorio 16

1. Introducción y Descripción General del Sistema

El presente documento técnico constituye la memoria descriptiva y formal de entrega correspondiente al Trabajo Práctico Integrador (TPI) de la asignatura Tecnologías de Desarrollo de Software IDE, dictada en la carrera de Ingeniería en Sistemas de Información de la Universidad Tecnológica Nacional (UTN - FRRo).
El producto de software desarrollado es el Sistema de Gestión Académica (SGA), una plataforma integral orientada a la administración docente, curricular y estudiantil de una institución universitaria. El sistema centraliza y automatiza los procesos esenciales de matriculación, inscripciones a cursados, gestión de cupos, asignación de comisiones y registro de calificaciones, proveyendo al mismo tiempo indicadores analíticos y reportes estadísticos para el seguimiento del rendimiento pedagógico institucional.
El desarrollo fue concebido bajo estándares profesionales de ingeniería de software, articulando una arquitectura en capas altamente desacoplada con backend WebAPI RESTful en .NET 8, doble interfaz de usuario (Escritorio en Windows Forms y Web Single Page Application en Blazor WebAssembly), esquema de autenticación y autorización robusto mediante tokens JWT, y una estrategia de persistencia híbrida que integra Entity Framework Core 8 con ADO.NET clásico.

2. Requerimientos del Sistema

2.1. Requerimientos Funcionales (RF)

Código	Requerimiento	Descripción Funcional
RF-01	**Autenticación Flexible de Usuarios**	El sistema provee una pantalla de inicio de sesión segura. Permite autenticarse mediante nombre de usuario, número de legajo o dirección de correo electrónico institucional, combinados con su contraseña cifrada. Tras la validación, el servidor emite un JSON Web Token (JWT) con los claims correspondientes.
RF-02	**Autorización y Control de Acceso por Roles**	Restricción dinámica de funcionalidades, menús y endpoints según el perfil del usuario autenticado: Administrador (acceso total a los 7 ABMs y módulos de configuración), Docente (acceso a cursos asignados, listados de inscriptos, carga de notas/condición y reportes) y Alumno (consulta de oferta académica, inscripción a cursados y visualización de historial de calificaciones).
RF-03	**Gestión Integral de Usuarios y Personas (ABM 1)**	Administración de cuentas y legajos del personal. Aplica una relación 1 a 1 normalizada donde los atributos personales (Nombre, Apellido, Dirección, Teléfono, Email, Fecha de Nacimiento) residen en Persona, mientras que las credenciales y rol residen en Usuario. El sistema asigna automáticamente el número de legajo incremental.

RF-04	**Gestión de Especialidades (ABM 2)**	Alta, baja, modificación y consulta de las carreras o especialidades académicas de la facultad (ej: Ingeniería en Sistemas de Información, Ingeniería Química).
RF-05	**Gestión de Planes de Estudio y Materias (ABM 3 - Patrón Maestro/Detalle)**	Permite estructurar los planes formativos y sus espacios curriculares dependientes aplicando el patrón relacional Maestro/Detalle en el formulario PlanDetalle: el Plan opera como cabecera/maestro (Especialidad y Nombre) y la nómina de Materias asociadas (con horas semanales y totales) actúa como detalle gestionado atómicamente.
RF-06	**Gestión de Materias Curriculares (ABM 4)**	Administración individual de los espacios curriculares pertenecientes a cada plan formativo, definiendo nombre, horas semanales y carga horaria total.
RF-07	**Gestión de Comisiones (ABM 5)**	Definición de divisiones de cursado organizadas por año de especialidad y plan de estudio (ej: Comisión 3K1, Comisión 3K2).
RF-08	**Gestión de Cursos (ABM 6)**	Apertura formal de cursos para un año calendario específico, asociando materia, comisión, cupo máximo disponible y docentes responsables.
RF-09	**Gestión de Inscripciones y Calificaciones (ABM 7)**	Ciclo completo de matriculación estudiantil con control preventivo de cupos disponibles. Permite a los docentes registrar la condición académica final (Aprobado, Regular, Libre) y la nota numérica definitiva.
RF-10	**Búsqueda Avanzada con Filtros Multicriterio**	Módulos de consulta de cursos e inscripciones que permiten el filtrado dinámico por texto, materia, comisión o año lectivo.
RF-11	**Reporte 1: Ocupación de Cursos y Análisis de Cupos**	Generación de reportes analíticos comparativos entre el Cupo Máximo configurado y los Inscriptos Reales por curso. Incluye gráfico de barras renderizado mediante GDI+ (System.Drawing) en la aplicación de escritorio y gráfico vectorial interactivo SVG en la aplicación Web.
RF-12	**Reporte 2: Rendimiento Académico por Materia**	Reporte tabular estadístico que clasifica y calcula los porcentajes relativos de alumnos Aprobados, Regulares y Libres por cada materia curricular, junto con la nota promedio obtenida.

2.2. Requerimientos No Funcionales (RNF)

Código	Requerimiento Técnico	Descripción No Funcional
RNF-01	**Arquitectura en Capas y Desacoplamiento**	Estructuración de la solución en capas desacopladas: Presentación (Desktop y Web), Biblioteca de Clientes HTTP compartida, Backend RESTful WebAPI, Capa de Servicios y Capa de Persistencia.
RNF-02	**Persistencia Híbrida (EF Core + ADO.NET)**	Uso prioritario de Entity Framework Core 8 para la abstracción ORM relacional orientada a objetos, e implementación directa de ADO.NET nativo (SqlConnection, SqlCommand, SqlDataReader) en el repositorio EspecialidadRepository.
RNF-03	**Autogeneración y Sembrado de Base de Datos**	Creación automática de la base de datos SQL Server LocalDB al inicio (Database.EnsureCreated()) con precarga de un set completo y verosímil de datos de prueba.
RNF-04	**Seguridad Robusta y Cifrado**	Comunicación segura cliente-servidor basada en estándar HTTP portando tokens JWT. Contraseñas de usuario cifradas con algoritmos seguros.
RNF-05	**Validaciones en Múltiples Capas**	Restricciones y consistencia de datos validadas tanto a nivel de formulario en la UI como en la capa de servicios de aplicación.
RNF-06	**Compatibilidad Multiplataforma / Dual-UI**	Operatividad consistente desde una aplicación de escritorio Windows Forms (.NET 8 Windows) y desde un navegador web moderno mediante Blazor WebAssembly (.NET 8).

3. Arquitectura del Sistema

El sistema fue diseñado adoptando una Arquitectura en N-Capas Orientada a Servicios (SOA) con separación estricta de responsabilidades. Toda la interacción con el repositorio de datos y las reglas del dominio se centraliza en el backend WebAPI, permitiendo que tanto la interfaz de escritorio Windows Forms como la interfaz web Blazor WebAssembly actúen como clientes delgados y desacoplados.

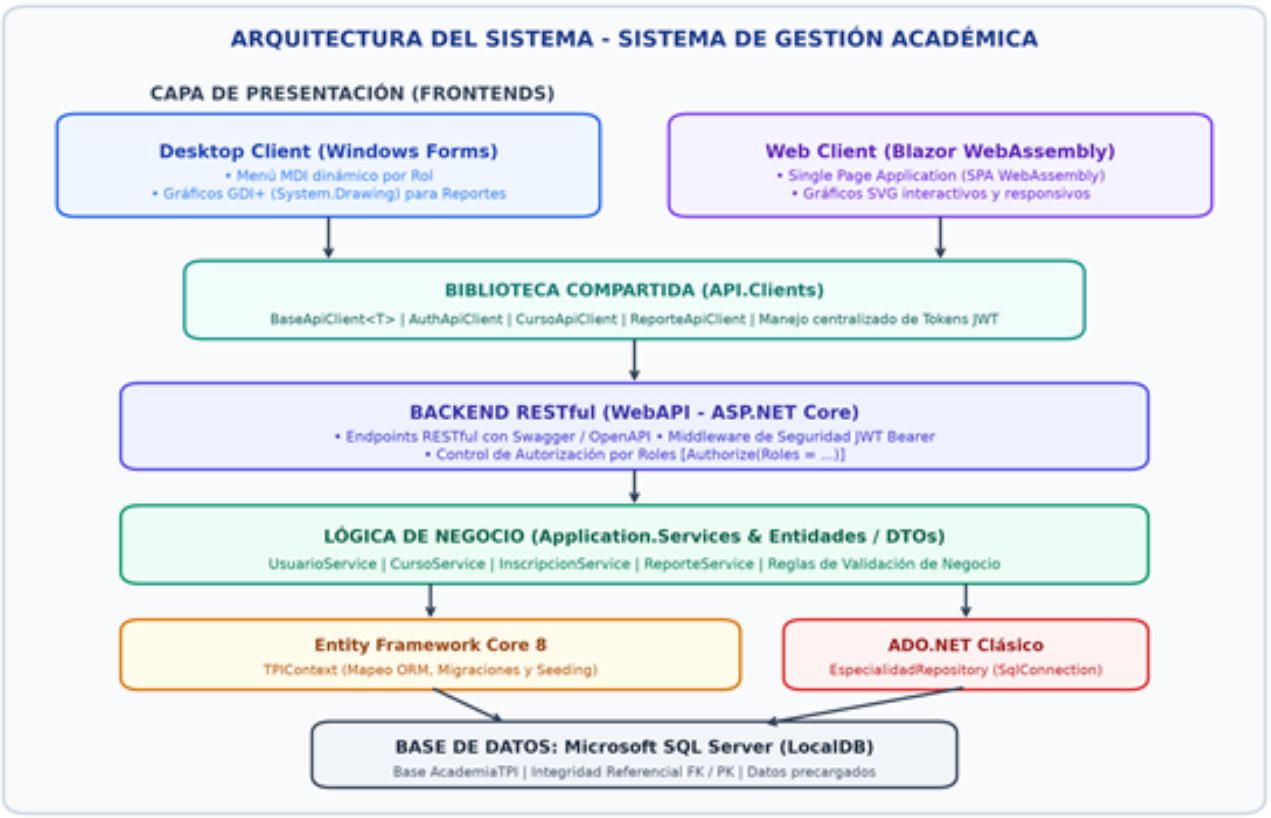


Figura 1: Diagrama de Arquitectura de la Solución Academia.

3.1. Detalle de Proyectos de la Solución

- **WebAPI (Servicio REST / Host):** Expone los controladores HTTP RESTful documentados mediante Swagger / OpenAPI. Gestiona la autenticación JWT Bearer, la resolución de Inyección de Dependencias y el ciclo de vida de la conexión a la base de datos.
- **Application.Services (Lógica de Negocio):** Contiene las clases de servicio (UsuarioService, CursoService, InscripcionService, ReporteService) donde se validan y ejecutan las reglas del negocio institucional (validación de cupos, asignación de legajos, cálculo de promedios).
- **Data (Acceso a Datos y Persistencia):** Implementa el DbContext de Entity Framework Core 8 (TPIContext) con las configuraciones Fluent API y sembrado de datos (Seed). Aloja adicionalmente el repositorio ADO.NET EspecialidadRepository.
- **Entidades (Modelo de Dominio):** Define las entidades del negocio: Persona, Usuario, Especialidad, Plan, Materia, Comision, Curso, DocenteCurso, AlumnoInscripcion.

- **DTOs (Transferencia de Datos):** Contiene los objetos planos (Data Transfer Objects) optimizados para serialización JSON segura entre la WebAPI y los clientes, evitando exponer directamente las entidades de base de datos.
- **API.Clients (Librería de Consumo HTTP):** Componente reutilizable compartido por ambos frontends. Provee la clase genérica BaseApiClient<T> y clientes especializados (AuthApiClient, CursoApiClient, ReporteApiClient) con inyección automática de cabeceras de autorización Bearer.
- **WindowsForms (Frontend de Escritorio):** Aplicación cliente nativa con arquitectura MDI (Multiple Document Interface). Permite la operación ágil de los 7 ABMs, filtros y renderizado de gráficos estadísticos con GDI+ (System.Drawing).
- **Blazor.WebAssembly (Frontend Web SPA):** Aplicación Web Single Page Application compilada a WebAssembly en el navegador. Provee una interfaz reactiva, adaptativa y moderna con gráficos SVG vectoriales sin dependencias de librerías externas.

4. Modelo de Datos

La base de datos relacional del sistema fue diseñada con el motor Microsoft SQL Server (instancia LocalDB bajo el nombre AcademiaTPI). El esquema relacional satisface estrictamente las formas normales de bases de datos relacionales, garantizando integridad referencial mediante claves foráneas con reglas de restricción ante eliminaciones en cascada para proteger datos académicos históricos.

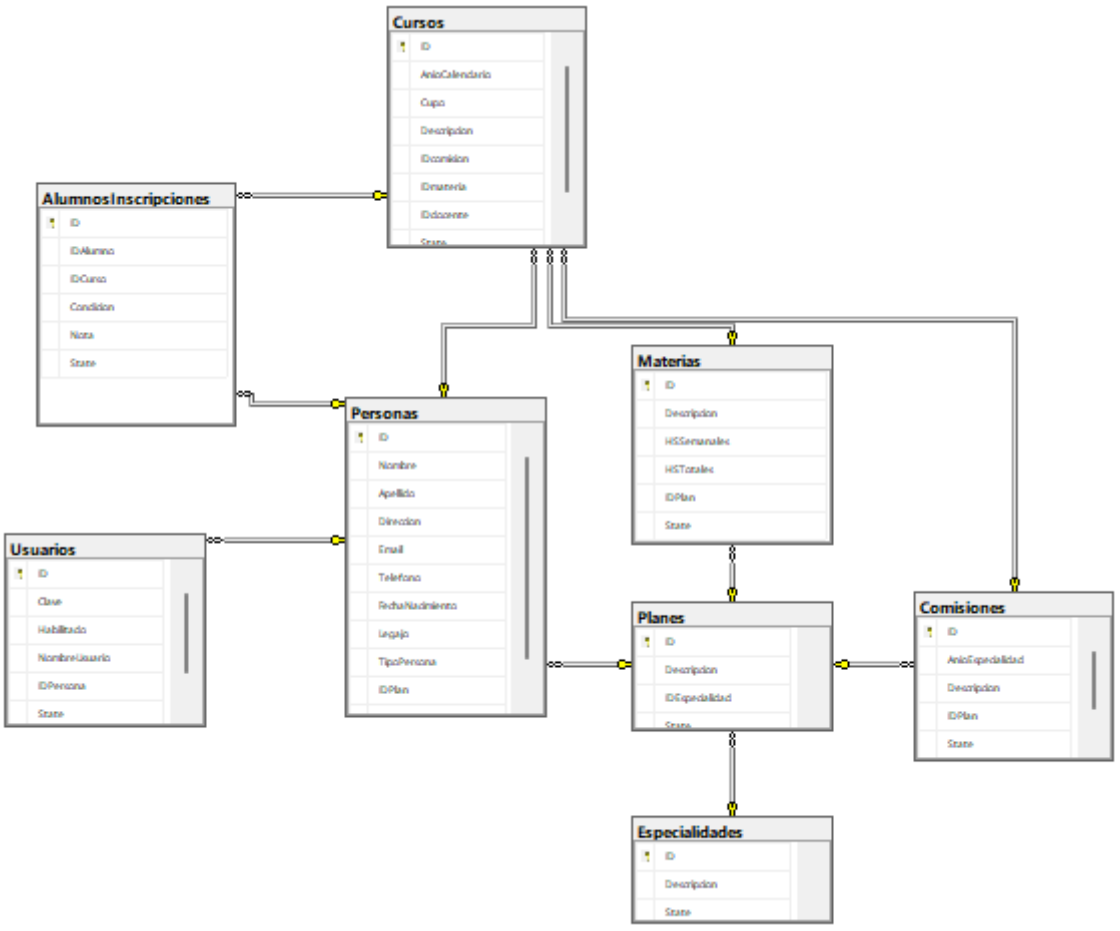
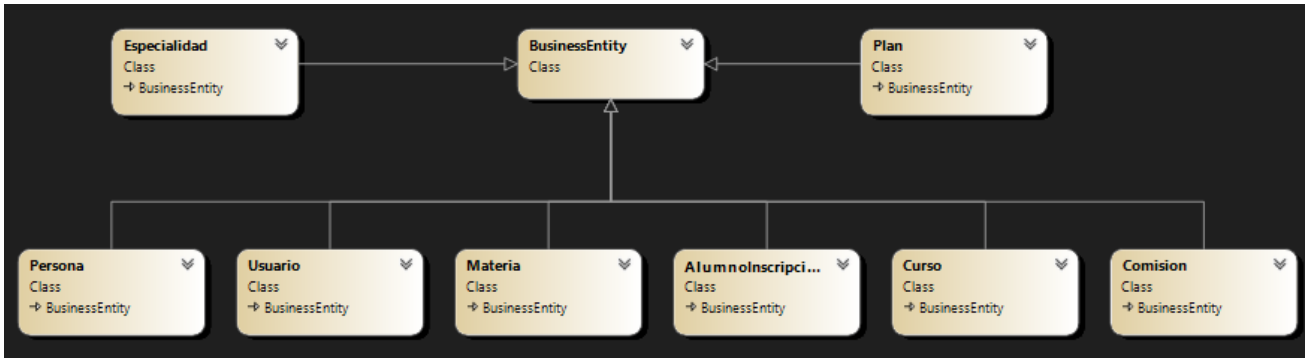


Figura 2: Diagrama Entidad-Relación (DER) del Sistema de Gestión Académica.

4.1. Modelo de Dominio (Diagrama de Clases UML)



4.2. Normalización y Relación Persona - Usuario

Conforme a las pautas de diseño y las observaciones pedagógicas de la cátedra, se eliminó toda redundancia entre las entidades Persona y Usuario. La información filiatoria y de contacto (Nombre, Apellido, Dirección, Teléfono, Email, Fecha de Nacimiento, Legajo y TipoPersona) reside exclusivamente en la tabla PERSONAS. La tabla USUARIOS mantiene una relación 1 a 1 mediante la clave foránea PersonaId, limitándose a almacenar el NombreUsuario, Clave cifrada, Estado Habilitado y Rol asignado.

4.3. Diccionario de Datos

Tabla	Estructura de Columnas y Restricciones
PERSONAS	Id (PK, int), Nombre (varchar 50), Apellido (varchar 50), Direccion (varchar 100), Email (varchar 100), Telefono (varchar 50), FechaNacimiento (date), TipoPersona (int), Legajo (int)
USUARIOS	Id (PK, int), NombreUsuario (varchar 50), Clave (varchar 100), Habilitado (bit), Rol (varchar 20), PersonaId (FK -> Personas.Id)
ESPECIALIDADES	Id (PK, int), Descripcion (varchar 100)
PLANES	Id (PK, int), Descripcion (varchar 100), EspecialidadId (FK -> Especialidades.Id)
MATERIAS	Id (PK, int), Descripcion (varchar 100), HsSemanales (int), HsTotales (int), PlanId (FK -> Planes.Id)
COMISIONES	Id (PK, int), Descripcion (varchar 100), AnioEspecialidad (int), PlanId (FK -> Planes.Id)
CURSOS	Id (PK, int), AnioCalendario (int), Cupo (int), Descripcion (varchar 100), MateriaId (FK -> Materias.Id), ComisionId (FK -> Comisiones.Id)
DOCENTES_CURSOS	Id (PK, int), Cargo (int), CursoId (FK -> Cursos.Id), DocenteId (FK -> Personas.Id)
ALUMNOS_INSCRIPCIONES	Id (PK, int), Condicion (varchar 50), Nota (int, nullable), AlumnoId (FK -> Personas.Id), CursoId (FK -> Cursos.Id)

5. Patrones de Diseño y Decisión Técnica de Persistencia

5.1. Implementación Obligatoria de ADO.NET Clásico

En estricto cumplimiento con los requerimientos técnicos de la cátedra ('Para el acceso a datos es necesario usar ADO.NET al menos una vez'), se diseñó el repositorio EspecialidadRepository dentro del proyecto Data utilizando las clases nativas de Microsoft.Data.SqlClient.

En dicho repositorio se gestionan las conexiones físicas directas mediante SqlConnection, la formulación y ejecución parametrizada de sentencias SQL (evitando inyecciones SQL) a través de SqlCommand, y la lectura eficiente de registros mediante SqlDataReader para las operaciones de consulta, inserción y modificación de especialidades.

5.2. Persistencia Principal con Entity Framework Core 8

Para el resto del ecosistema relacional (Cursos, Inscripciones, Materias, Planes, Comisiones y Usuarios), se implementó Entity Framework Core 8. Esto proporciona las siguientes ventajas operativas y de ingeniería:

- Mapeo Objeto-Relacional (ORM) robusto con LINQ para consultas fuertemente tipadas y seguras en tiempo de compilación.
- Carga Eager (Eager Loading) mediante .Include() y .ThenInclude() para resolver grafos de navegación complejos (ej: Curso -> Materia -> Plan -> Especialidad).
- Estrategia de Sembrado Automatizado (Database.EnsureCreated()): la primera vez que se ejecuta la solución, el TPIContext crea el esquema de la base en SQL Server LocalDB y carga un dataset representativo de docentes, alumnos, cursos e inscripciones listo para pruebas.

5.3. Patrón Maestro/Detalle (Planes de Estudio y Materias)

El patrón relacional Maestro/Detalle exigido por la cátedra fue implementado en el módulo de Planes de Estudio a través del formulario PlanDetalle: el Plan de Estudio actúa como entidad cabecera o Maestro (Especialidad y Nombre del Plan), mientras que la grilla de Materias asociadas (con sus Horas Semanales y Horas Totales) opera como Detalle.

Las materias asociadas son manipuladas en memoria dentro de una lista local (materiasLocales), permitiendo agregar nuevas materias, modificarlas o eliminarlas antes de efectuar la persistencia atómica definitiva hacia el backend y la base de datos.

6. Tecnologías y Herramientas Utilizadas

Componente	Tecnología Seleccionada	Justificación y Rol en la Solución
Lenguaje de Programación	**C# 12**	Lenguaje fuertemente tipado, orientación a objetos moderna, pattern matching y records.
Plataforma / Framework	**.NET 8.0 SDK**	LTS (Long Term Support), alto rendimiento multiplataforma.

Entorno de Desarrollo (IDE)	**Visual Studio 2022 / VS Code**	Desarrollo, depuración integrada y gestión de soluciones multicapa.
Motor de Base de Datos	**Microsoft SQL Server 2022 LocalDB**	Persistencia relacional de fácil portabilidad local sin configuración manual.
ORM / Acceso a Datos	**Entity Framework Core 8.0 & ADO.NET**	EF Core para navegación relacional y SqlClient directo en EspecialidadRepository.
Framework Backend	**ASP.NET Core WebAPI**	Arquitectura RESTful, inyección de dependencias nativa y middleware modular.
Documentación de API	**Swagger UI / Swashbuckle OpenAPI**	Exploración interactiva y prueba en tiempo real de los endpoints REST.
Seguridad y Autenticación	**JWT (JSON Web Tokens) & BCrypt**	Cifrado seguro de credenciales y autorización basada en Claims/Roles.
Cliente de Escritorio	**Windows Forms (.NET 8)**	Interfaz nativa MDI, controles DataGridView desacoplados y gráficos con GDI+.
Cliente Web	**Blazor WebAssembly (.NET 8)**	Single Page Application ejecutada en el navegador vía WebAssembly, gráficos en SVG.
Control de Versiones	**Git / GitHub**	Repositorio distribuido con historial trazable de commits y ramas colaborativas.

7. Datos de Prueba y Manual de Credenciales

Para facilitar la evaluación y corrección del sistema por parte del cuerpo docente, la base de datos se autogenera con un dataset preconfigurado de acuerdo a la regla de negocio del sistema: el nombre de usuario es el primer nombre en minúsculas y la clave corresponde a su número de legajo.

7.1. Administrador

• Usuario: admin | Contraseña: admin123 | Alcance: Acceso irrestricto a los 7 ABMs y a ambos Reportes.

7.2. Profesores (Docentes)

Acceso a cursos a su cargo, calificación de alumnos inscriptos y consulta de ambos Reportes.

Docente	Usuario	Contraseña	Email	Cursos Asignados
Ing. Carlos Gómez	**carlos**	**2001**	carlos.gomez@academia.utn.edu.ar	Desarrollo .NET (Tarde y Noche), Sistemas Operativos
Dra. Laura Martínez	**laura**	**2002**	laura.martinez@academia.utn.edu.ar	Bases de Datos, Ingeniería de Software
Lic. Roberto Sánchez	**roberto**	**2003**	roberto.sanchez@academia.utn.edu.ar	Algoritmos y Estructuras de Datos

7.3. Alumnos

Acceso a matriculación e inscripción a cursados, consulta de materias y visualización de notas.

Alumno	Usuario	Contraseña	Email Institucional
Juan Pérez	**juan**	**5001**	juan.perez@academia.utn.edu.ar
María López	**maria**	**5002**	maria.lopez@academia.utn.edu.ar
Lucas Fernández	**lucas**	**5003**	lucas.fernandez@academia.utn.edu.ar
Sofía Rossi	**sofia**	**5004**	sofia.rossi@academia.utn.edu.ar
Mateo Benítez	**mateo**	**5005**	mateo.benitez@academia.utn.edu.ar
Camila Rodríguez	**camila**	**5006**	camila.rodriguez@academia.utn.edu.ar
Tomás Díaz	**tomas**	**5007**	tomas.diaz@academia.utn.edu.ar
Valentina Romero	**valentina**	**5008**	valentina.romero@academia.utn.edu.ar

8. Evidencias de Ejecución

A continuación se incorporan las evidencias visuales del sistema en funcionamiento real, abarcando los clientes de Escritorio (Windows Forms) y Web (Blazor WebAssembly), los 7 ABMs, el patrón Maestro/Detalle y los dos reportes estadísticos obligatorios.

8.1. Aplicación de Escritorio (Windows Forms)

• Pantalla de Inicio de Sesión (Login)

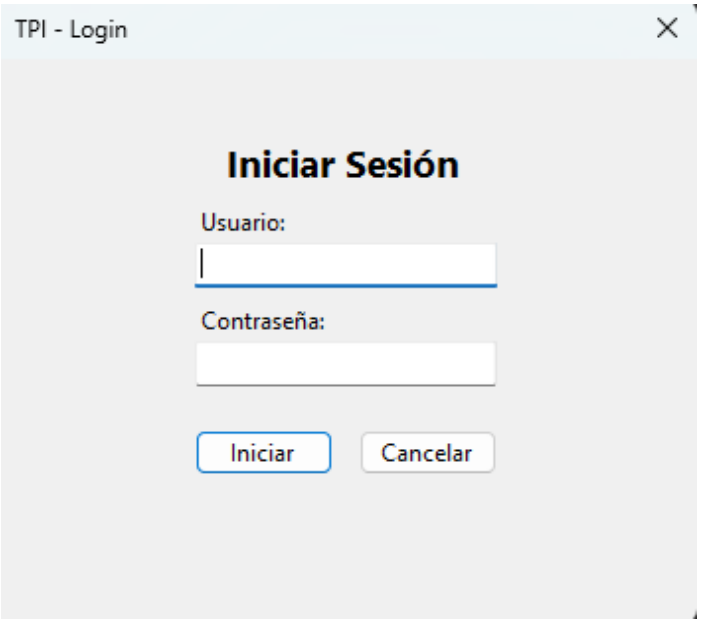


Figura: Formulario de inicio de sesión de escritorio permitiendo ingreso por usuario, legajo o email.

• Menú Principal MDI y Listado de Cursos con Filtro

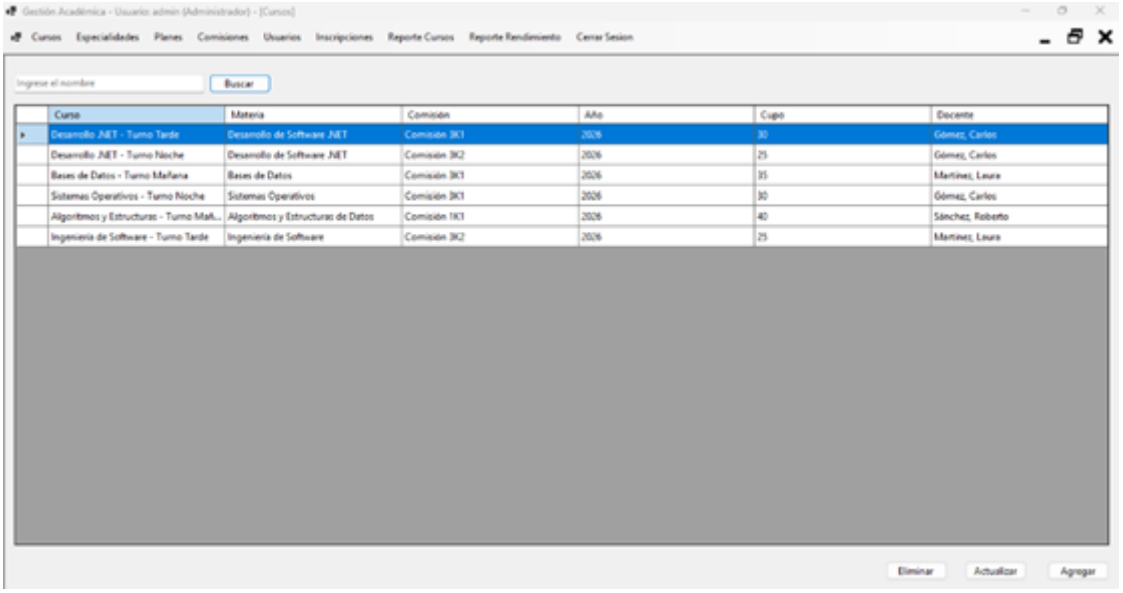


Figura: Ventana principal con arquitectura MDI, barra de navegación superior adaptada al rol Administrador y buscador dinámico de cursos.

- **Gestión de Planes de Estudio y Materias (Patrón Maestro/Detalle)**

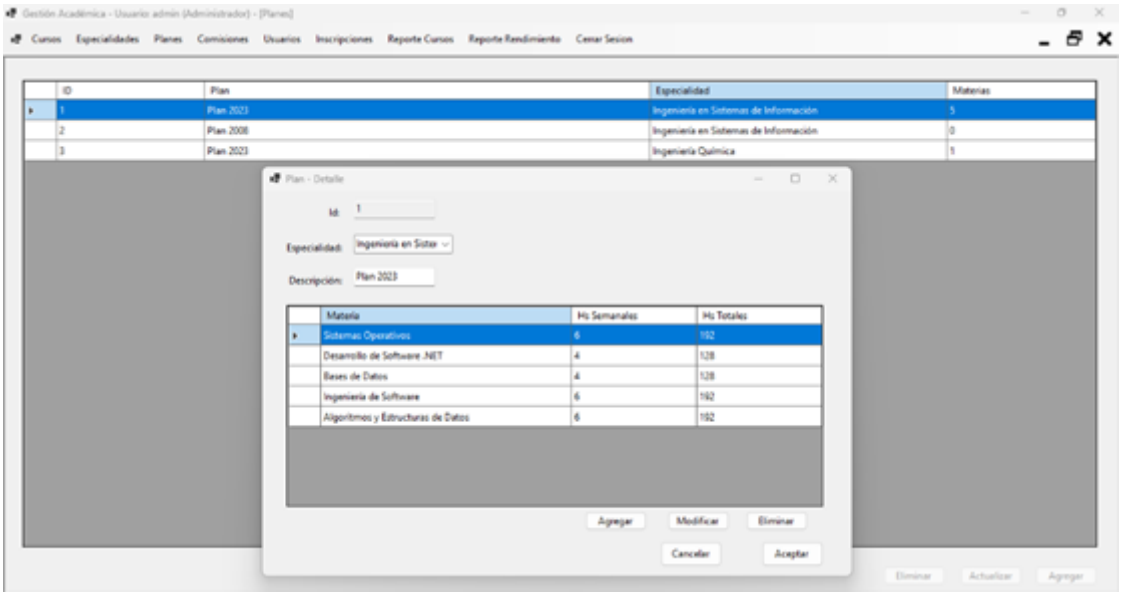


Figura: Formulario PlanDetalle: cabecera del Plan (Especialidad y Nombre) y grilla inferior con el detalle de materias asociadas.

- **Reporte 1: Ocupación de Cursos (Gráfico de Barras GDI+)**

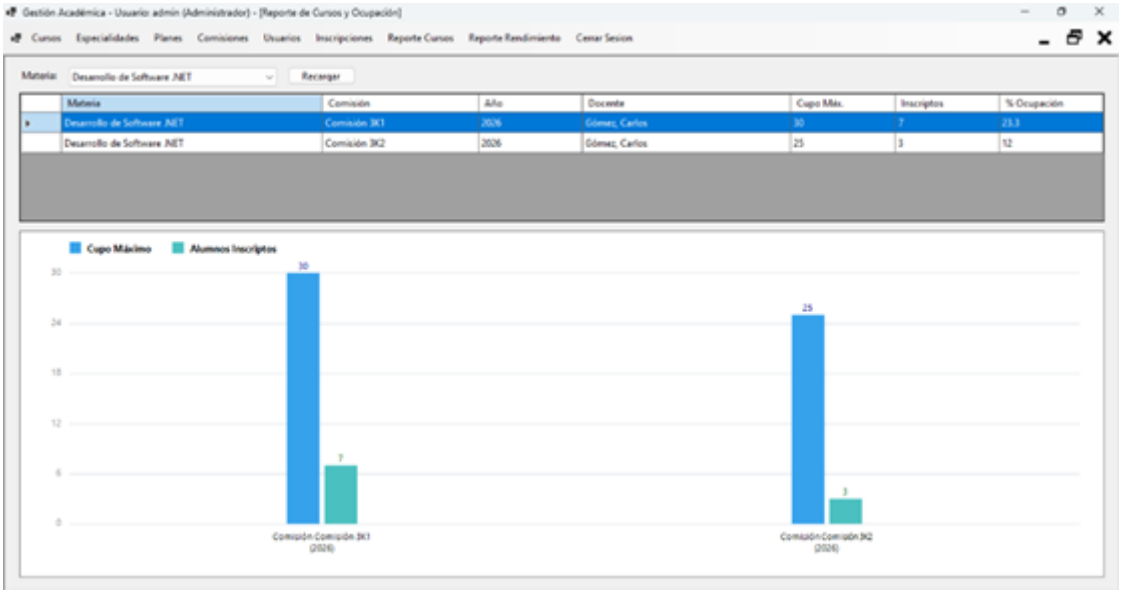


Figura: Reporte comparativo de Cupo Máximo vs. Alumnos Inscriptos renderizado con GDI+ (System.Drawing) y porcentaje de ocupación.

- **Reporte 2: Rendimiento Académico por Materia**

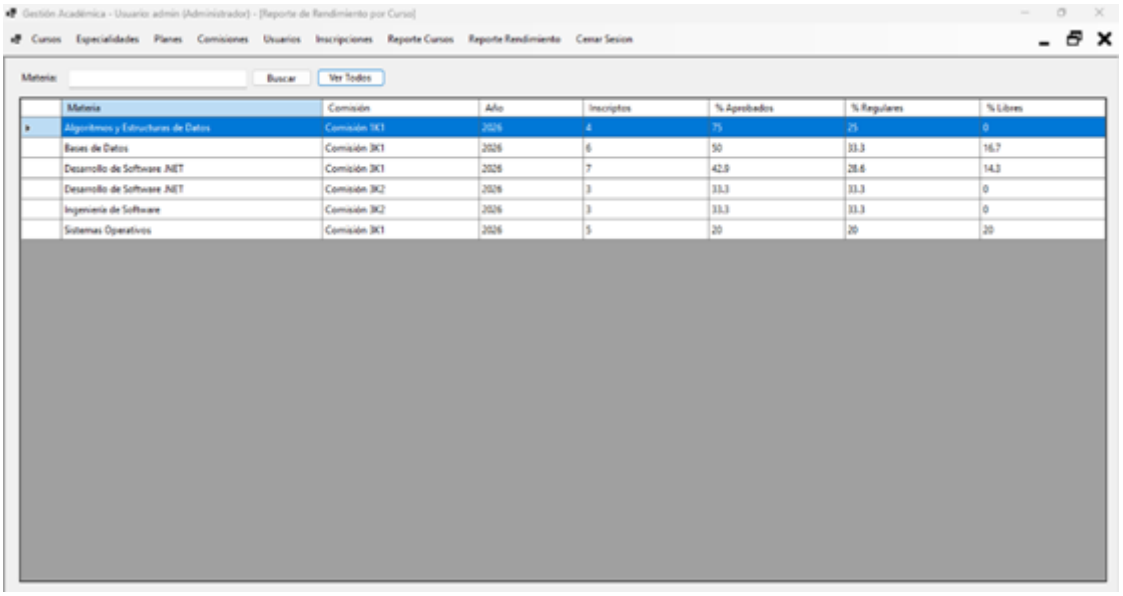


Figura: Reporte tabular estadístico con desglose porcentual de alumnos Aprobados, Regulares y Libres por curso y materia.

8.2. Aplicación Web (Blazor WebAssembly)

- **Interfaz Web SPA y Gestión de Cursos**

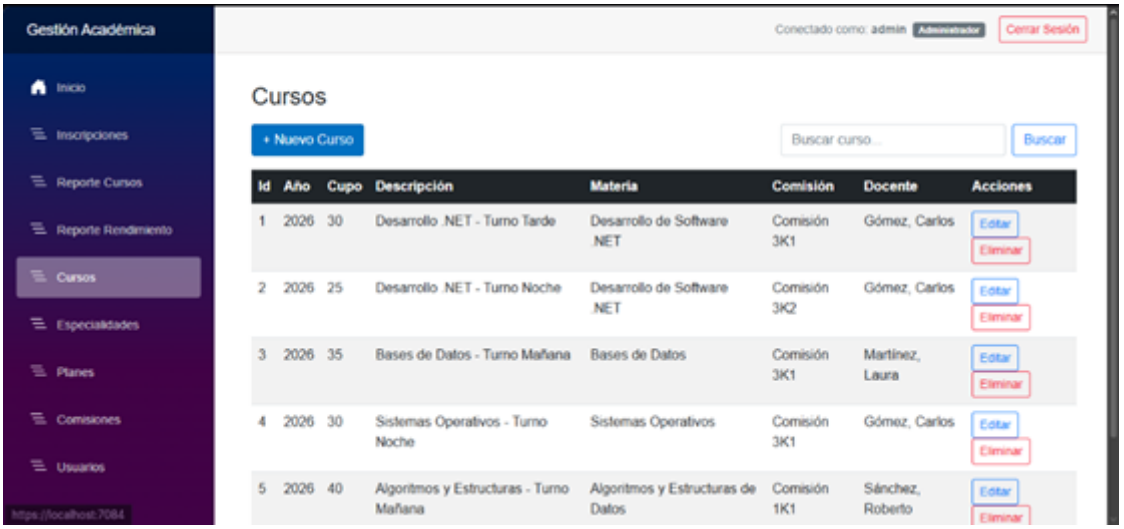


Figura: Cliente Web Blazor WebAssembly con menú lateral reactivo, usuario conectado y tabla de cursos.

- **Reporte 1 Web: Ocupación de Cursos (Gráfico Vectorial SVG)**

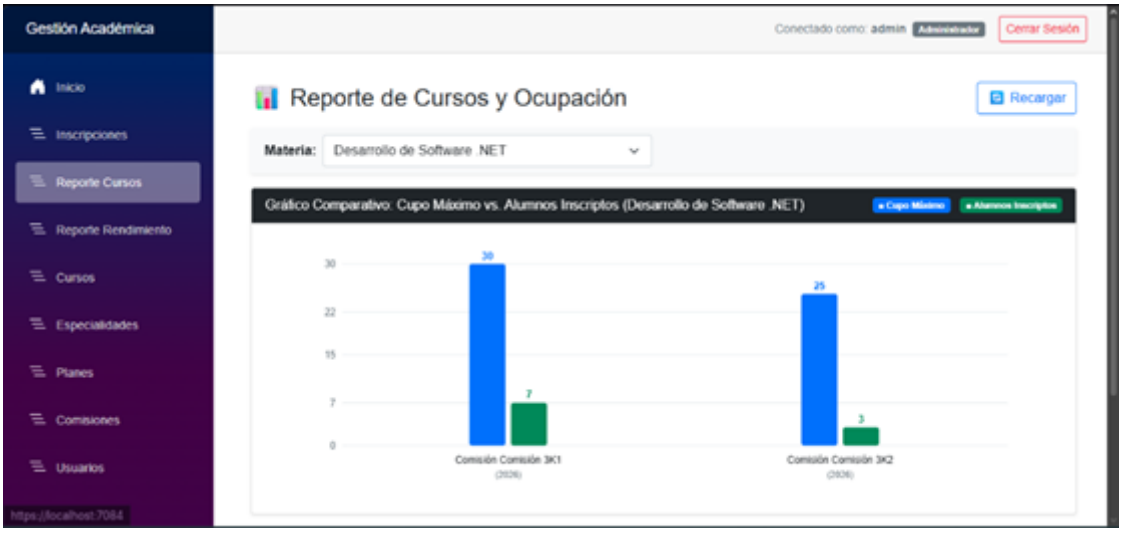


Figura: Gráfico comparativo de barras vectoriales SVG interactivo y responsive con selector de materias.

- **Reporte 2 Web: Rendimiento Académico por Curso**



Materia	Comisión	Año	Docente	Inscriptos	Aprobados	% Aprobados	Regulares	% Regulares	Libres	% Libres	% Cursando	Nota Prom.
Algoritmos y Estructuras de Datos	Comisión 1K1	2026	Sánchez, Roberto	4	3	75%	1	25%	0	0%	0%	7,50
Bases de Datos	Comisión 3K1	2026	Martínez, Laura	6	3	50%	2	33,3%	1	16,7%	0%	7,17
Desarrollo de Software .NET	Comisión 3K1	2026	Gómez, Carlos	7	3	42,9%	2	28,6%	1	14,3%	14,3%	6,17
Desarrollo												

Figura: Tabla dinámica de rendimiento en Blazor mostrando porcentajes de aprobación, regularidad, condición libre y nota promedio.

9. Conclusiones y Enlaces del Repositorio

El desarrollo del Sistema de Gestión Académica permitió aplicar de manera integral las directrices pedagógicas y tecnológicas de la cátedra. Se consolidó una arquitectura desacoplada basada en micro-responsabilidades, garantizando alta cohesión, facilidad de mantenimiento y extensibilidad para futuros requerimientos institucionales.

Se verificó satisfactoriamente el funcionamiento simultáneo de ambos clientes (Escritorio y Web) interactuando con el backend RESTful bajo autorización JWT, así como la coexistencia armónica de Entity Framework Core 8 con ADO.NET clásico.

ENLACE AL REPOSITORIO DE CÓDIGO FUENTE: El código fuente completo de la solución se encuentra alojado en el repositorio público de GitHub:

https://github.com/Simon-Yocco/Academia.git

El repositorio incluye las instrucciones completas de clonación, compilación y ejecución inmediata sin configuración previa.